

Idea for my bachelor's thesis: *VLM in Flash*

Seol Handong

Under the displayed upper-bound formulation, an exact optimum can always be represented by a single contiguous chunk.

Problem setup

Let N be the number of input channels of a weight matrix and let

$$M \in \{0, 1\}^N,$$

where $M_i = 1$ means that channel i and its corresponding weight row are selected.

Let $\mathcal{C}(M)$ denote the set of maximal contiguous runs of selected channels.

Define

$$R(M) = \|M\|_1$$

as the number of selected rows and

$$K(M) = |\mathcal{C}(M)|$$

as the number of maximal contiguous chunks. Under the paper's additive latency model,

$$L_{\text{table}}(M) = \sum_{C \in \mathcal{C}(M)} T[|C|],$$

where $T[r]$ is the profiled flash-read latency associated with a contiguous chunk of r rows.

Define the total retained activation importance as

$$I(M) = \sum_{i=0}^{N-1} v_i M_i,$$

Assume $v_i \geq 0$, $I_{\text{tot}} = \sum_i v_i > 0$, $T[r] > 0$, and $1 \leq R \leq N$. The paper's displayed selection problem can then be written as

$$\boxed{\max_{\substack{M \in \{0,1\}^N \\ 1 \leq R(M) \leq R}} \frac{I(M)}{L_{\text{table}}(M)}}, \quad R = \lfloor (1 - \rho)N \rfloor.$$

The lower bound excludes the empty mask, for which the ratio is undefined.

For a contiguous chunk C , define

$$I(C) = \sum_{i \in C} v_i, \quad U(C) = \frac{I(C)}{T[|C|]}.$$

Here $U(C)$ is the chunk's retained importance per unit of estimated flash-read latency.

Single-chunk optimality

Because distinct maximal chunks partition the selected channels,

$$I(M) = \sum_{C \in \mathcal{C}(M)} I(C).$$

Therefore,

$$\frac{I(M)}{L_{\text{table}}(M)} = \frac{\sum_{C \in \mathcal{C}(M)} I(C)}{\sum_{D \in \mathcal{C}(M)} T[|D|]} = \sum_{C \in \mathcal{C}(M)} \underbrace{\frac{T[|C|]}{\sum_{D \in \mathcal{C}(M)} T[|D|]}}_{w_C} \underbrace{\frac{I(C)}{T[|C|]}}_{U(C)}.$$

The coefficients satisfy

$$w_C \geq 0, \quad \sum_{C \in \mathcal{C}(M)} w_C = 1.$$

Hence the mask-level importance–latency ratio is a latency-weighted average of the utilities of its constituent chunks:

$$\boxed{\frac{I(M)}{L_{\text{table}}(M)} \leq \max_{C \in \mathcal{C}(M)} U(C)}.$$

Let C^* be the highest-utility contiguous interval whose length does not exceed R :

$$C^* \in \arg \max_{\substack{C \text{ contiguous} \\ 1 \leq |C| \leq R}} \frac{I(C)}{T[|C|]}.$$

The mask selecting only C^* is feasible and attains

$$\frac{I(C^*)}{T[|C^*|]} = U(C^*).$$

Consequently,

$$\boxed{\max_{\substack{M \in \{0,1\}^N \\ 1 \leq R(M) \leq R}} \frac{I(M)}{L_{\text{table}}(M)} = \max_{\substack{C \text{ contiguous} \\ 1 \leq |C| \leq R}} U(C)}.$$

Thus, under the displayed objective and additive latency assumption, there always exists an exact optimum consisting of only one contiguous chunk. Multiple chunks can also be optimal in the exceptional case where every selected chunk has the same maximum utility.

Exact search with prefix sums

This optimum can be found without examining all 2^N masks. Define the importance prefix sums

$$P_0 = 0, \quad P_j = \sum_{i=0}^{j-1} v_i.$$

A chunk

$$C_{i,\ell} = \{i, i+1, \dots, i+\ell-1\}$$

then has

$$I(C_{i,\ell}) = P_{i+\ell} - P_i$$

and utility

$$U(i, \ell) = \frac{P_{i+\ell} - P_i}{T[\ell]}.$$

There are

$$\sum_{\ell=1}^R (N - \ell + 1) = \frac{R(2N - R + 1)}{2}$$

contiguous intervals of length at most R . Without the length restriction, the total number is

$$\binom{N+1}{2} = \frac{N(N+1)}{2}.$$

Prefix sums require $O(N)$ preprocessing and make each interval score computable in $O(1)$. The exact optimum is therefore obtainable in

$$\boxed{O(NR)}$$

time and $O(N)$ memory, which becomes $O(N^2)$ time in the worst case $R = \Theta(N)$.

Mismatch between the formulation and implementation

This simple exact solution exposes a mismatch between the displayed formulation and the implemented algorithm. The displayed constraint

$$R(M) \leq R$$

permits the optimizer to leave most of the budget unused, whereas the paper's greedy procedure continues accepting chunks toward R . Its implemented behavior is therefore closer to a fixed-cardinality or cardinality-band problem such as

$$R(M) = R$$

or

$$R_- \leq R(M) \leq R_+,$$

for which the maximum-utility single-chunk argument no longer provides a general exact solution.

A fixed-budget objective captures the budget-filling behavior of the greedy algorithm.

Fixed-budget interpretation

The paper's implementation treats

$$R = \lfloor (1 - \rho)N \rfloor$$

as a target number of weight rows to retain. Rather than deciding the sparsity level, the selection algorithm receives R from the sparsity allocation and determines which rows should be loaded.

Let $\mathcal{G}$ denote the generated set of contiguous candidate chunks. Each candidate $C \in \mathcal{G}$ is assigned the utility

$$U(C) = \frac{I(C)}{T[|C|]}, \quad I(C) = \sum_{i \in C} v_i.$$

Thus, $U(C)$ measures the activation importance preserved by the chunk per unit of estimated flash-read latency.

Greedy selection rule

Let $G = |\mathcal{G}|$ be the number of generated candidates. They are sorted by decreasing utility:

$$U(C_{(1)}) \geq U(C_{(2)}) \geq \dots \geq U(C_{(G)}).$$

The algorithm initializes

$$S \leftarrow \emptyset, \quad q \leftarrow 0,$$

where S is the set of accepted chunks and q is the number of selected rows. It then processes the sorted candidates in order. A candidate $C_{(j)}$ is accepted if

$$C_{(j)} \cap \bigcup_{D \in S} D = \emptyset$$

and

$$q + |C_{(j)}| \leq R.$$

Upon acceptance,

$$S \leftarrow S \cup \{C_{(j)}\}, \quad q \leftarrow q + |C_{(j)}|.$$

The procedure continues until $q = R$ or no remaining candidate can be accepted. The resulting mask is

$$M_i^{\text{greedy}} = \mathbf{1} \left\{ i \in \bigcup_{C \in S} C \right\}.$$

The algorithm may return fewer than R rows when the remaining budget cannot be filled by any nonoverlapping candidate. Nevertheless, its operational goal is to select approximately R rows rather than to determine the row count from scratch.

Relationship to the global ratio

The candidate utility is the chunk-level counterpart of the mask-level objective

$$\frac{I(M)}{L_{\text{table}}(M)} = \frac{\sum_{C \in \mathcal{C}(M)} I(C)}{\sum_{C \in \mathcal{C}(M)} T[|C|]}.$$

Accordingly, the algorithm gives priority to chunks with high importance per estimated latency and greedily combines them while attempting to fill the prescribed row budget. This is a heuristic construction: ranking chunks individually does not guarantee that their combination maximizes the global ratio.

Comparison with top- R sparsification

The position of Neuron Chunking can be understood by comparing it with conventional top- R sparsification. Given the same prescribed row count, top- R selects

$$M_{\text{top-}R} \in \arg \max_{\substack{M \in \{0,1\}^N \\ R(M)=R}} I(M).$$

Because

$$I(M) = \sum_i v_i M_i,$$

this selects the R channels with the largest individual importance values. It maximizes retained activation importance without considering the physical arrangement or flash-read latency of the selected rows.

To study the budget-filling behavior of Neuron Chunking, we use the reconstructed benchmark

$$\max_{\substack{M \in \{0,1\}^N \\ R(M)=R}} \frac{I(M)}{L_{\text{table}}(M)}.$$

Top- R maximizes importance at a fixed row count; this benchmark balances importance against flash latency at the same row count.

Displayed constraint versus reconstructed objective

The paper formally uses $R(M) \leq R$. The fixed- R problem is our comparison benchmark, not a proved approximation guarantee for greedy. If greedy selects fewer than R rows, compare it at its actual row count $R(M_{\text{greedy}})$.

The profiled chunk latency is closely approximated by an affine function.

Affine fit of the released latency profiles

The paper represents the latency of reading a contiguous chunk using the profiled lookup table

$$T[r],$$

where r is the number of weight rows in the chunk. The released profiles instead record chunk size in kibibytes. Let z denote the chunk size in KiB. A least-squares fit to the released latency tables gives

$$T_{\text{KiB}}(z) \approx a + c_{\text{KiB}}z.$$

The fitted models are:

Device	Profiled range	Affine fit, with z in KiB and T in ms	R^2
Jetson Orin AGX	1–255 KiB	$T(z) \approx 0.01100 + 0.0001398z$	0.9867
Jetson Orin Nano	1–350 KiB	$T(z) \approx 0.01573 + 0.0002842z$	0.9940

These R^2 values are not reported in the original paper. They are obtained by fitting the latency tables released with its implementation. The mean absolute percentage errors are approximately 4.22% on AGX and 3.21% on Nano.

The maximum entrywise relative errors are 28.63% on AGX and 23.41% on Nano, so the fit is an average approximation. Its accuracy is not established beyond the profiled range; longer chunks selected by the DP require additional profiling. Here KiB means 1024 bytes, labeled “KB” in the implementation.

Fit in the throughput domain

The fit can also be examined in the throughput domain. The released implementation computes logical throughput as

$$\beta(z) = \frac{1000z}{1024T_{\text{KiB}}(z)} \text{ MiB/s},$$

where z is measured in KiB and $T_{\text{KiB}}(z)$ in milliseconds. Substituting the affine latency model gives

$$\hat{\beta}(z) = \frac{1000z}{1024(a + c_{\text{KiB}}z)}.$$

This function increases with chunk size and satisfies

$$\lim_{z \rightarrow \infty} \hat{\beta}(z) = \frac{1000}{1024c_{\text{KiB}}} \text{ MiB/s}.$$

It therefore reproduces the qualitative behavior observed in the paper: throughput rises rapidly for small contiguous reads and gradually approaches a bandwidth ceiling.

The throughput curves derived from the affine model also closely match the released measurements. Their coefficients of determination are

$$R_{\text{AGX}}^2 = 0.9875, \quad R_{\text{Nano}}^2 = 0.9866.$$

At the saturation sizes reported in Appendices D and H,

$$z_{\text{sat}}^{\text{AGX}} = 236 \text{ KiB}, \quad z_{\text{sat}}^{\text{Nano}} = 348 \text{ KiB},$$

the measured and affine-predicted throughputs are approximately:

Device	Measured throughput	Affine prediction
AGX	5221 MiB/s	5238 MiB/s
Nano	3096 MiB/s	2965 MiB/s

The fitted asymptotic throughputs are

$$\beta_{\infty}^{\text{AGX}} \approx 6985 \text{ MiB/s}, \quad \beta_{\infty}^{\text{Nano}} \approx 3436 \text{ MiB/s}.$$

For comparison, the advertised SSD limits are

$$7450 \text{ MB/s} \approx 7105 \text{ MiB/s}$$

for AGX and

$$3500 \text{ MB/s} \approx 3338 \text{ MiB/s}$$

for Nano. The fitted asymptotes differ from these advertised limits by approximately 1.7% and 2.9%, respectively.

This throughput check transforms the same latency data, so it is not independent validation. At the reported saturation sizes, the affine predictions reach only 75.0% and 86.3% of their asymptotes. The finite measured saturation points therefore differ from the model's extrapolated limits.

Conversion from kibibytes to weight rows

To express the model in weight-row units, suppose one weight row occupies b KiB. A chunk containing r rows then occupies

$$z = br \text{ KiB}.$$

Therefore,

$$T[r] = T_{\text{KiB}}(br) \approx a + c_{\text{KiB}} br.$$

Defining

$$c = bc_{\text{KiB}},$$

the row-based latency model becomes

$$T[r] \approx a + cr.$$

Here:

- a is the effective fixed latency incurred once per contiguous chunk;
- c is the marginal latency of adding one more weight row to that chunk.

The parameter a summarizes the effective penalty associated with small independent reads. It is inferred from the affine fit rather than separately measured by the paper as the cost of one particular hardware operation.

Consequences for mask-level latency

Define a separate affine cost, with $a, c > 0$:

$$L_{\text{aff}}(M) = \sum_{C \in \mathcal{C}(M)} (a + c|C|) = aK(M) + cR(M).$$

Thus $L_{\text{table}}(M) \approx L_{\text{aff}}(M)$. The affine cost depends only on the number of chunks and selected rows.

At a fixed row count, the transfer term cR is constant, so

$$R(M_1) = R(M_2) = R \implies [L_{\text{aff}}(M_1) < L_{\text{aff}}(M_2) \iff K(M_1) < K(M_2)].$$

Fewer chunks therefore mean lower affine latency. This exact ordering need not hold for the lookup table, whose costs also depend on individual chunk lengths.

The fixed-budget affine objective admits an exact polynomial-time dynamic program.

Fixed-budget formulation and DP state

Substituting the affine latency model into the reconstructed fixed-budget objective gives

$$\max_{\substack{M \in \{0,1\}^N \\ R(M)=R}} \frac{I(M)}{aK(M) + cR}.$$

Although there are 2^N possible masks, exhaustive enumeration is unnecessary. The selected channels form a one-dimensional binary sequence, and the number of chunks can be written as

$$K(M) = M_1 + \sum_{i=2}^N (1 - M_{i-1})M_i.$$

A new chunk begins exactly when a selected channel follows either the beginning of the sequence or an unselected channel. Consequently, the effect of selecting channel i depends only on whether channel $i - 1$ was selected. This local dependency allows dynamic programming over the channel chain.

For notational convenience, index the channel importances by $v_1, \dots, v_N$. Define

$$F_i^s(r, k)$$

as the maximum importance obtainable from the first i channels when

- exactly r channels have been selected;
- exactly k chunks have been formed;
- channel i has selection state $s \in \{0, 1\}$.

The initial conditions are

$$F_0^0(0, 0) = 0,$$

with every other initial or unreachable state set to $-\infty$.

Dynamic-programming recurrence

If channel i is not selected, the previous sequence may end in either state:

$$F_i^0(r, k) = \max \{ F_{i-1}^0(r, k), F_{i-1}^1(r, k) \}.$$

If channel i is selected, it either continues an existing chunk or starts a new one:

$$F_i^1(r, k) = v_i + \max \{ F_{i-1}^1(r-1, k), F_{i-1}^0(r-1, k-1) \}.$$

The maximum importance achievable with exactly R selected rows and k chunks is therefore

$$I^*(R, k) = \max_{s \in \{0,1\}} F_N^s(R, k).$$

The exact optimum of the affine fixed-budget problem is

$$\max_{1 \leq k \leq K_{\max}} \frac{I^*(R, k)}{ak + cR},$$

where

$$K_{\max} = \min(R, N - R + 1)$$

is the largest possible number of chunks in a binary mask containing exactly R selected positions.

Exact importance–latency frontier

The dynamic program yields more than the single ratio-maximizing mask. For every feasible chunk count k , it gives the largest importance achievable with exactly R selected rows. Since the affine latency associated with that state is

$$L_R(k) = ak + cR,$$

the points

$$\mathcal{F}_R = \{(ak + cR, I^*(R, k)) : 1 \leq k \leq K_{\max}\}$$

trace the exact fixed- R importance–latency trade-off under the affine model.

Increasing k strictly increases affine latency, but the exact- k values $I^*(R, k)$ need not increase: their feasible sets are not nested. A monotone envelope is

$$I_{\leq}^*(R, k) = \max_{1 \leq j \leq k} I^*(R, j).$$

The nondominated points in $\mathcal{F}_R$ form the fixed-budget Pareto frontier. The paper's greedy result,

$$(L_{\text{aff}}(M_{\text{greedy}}), I(M_{\text{greedy}})),$$

can be plotted against this frontier. This reveals how far the greedy mask lies from the best importance achievable at comparable affine latency. If the greedy procedure underfills the budget, it should instead be compared with the frontier corresponding to its realized row count $R(M_{\text{greedy}})$.

The dynamic program has time complexity

$$O(NRK_{\max})$$

and rolling-array memory complexity

$$O(RK_{\max}).$$

This memory bound covers values; storing all parent pointers for backtracking takes $O(NRK_{\max})$ memory.

Since $R \leq N$ and $K_{\max} \leq N$, this is polynomial rather than exponential, with a worst-case time complexity of $O(N^3)$. The recurrence therefore provides an exact oracle in principle. For full-scale projections, however, its practical use requires either reduced instances or further acceleration.

Faster solution through fractional programming

A faster formulation eliminates the explicit chunk-count dimension through fractional programming. For a parameter $\eta \geq 0$, define

$$\Psi(\eta) = \max_{\substack{M \in \{0,1\}^N \\ R(M)=R}} [I(M) - \eta (aK(M) + cR)].$$

Let

$$\eta^* = \max_{\substack{M \in \{0,1\}^N \\ R(M)=R}} \frac{I(M)}{aK(M) + cR}.$$

Then

$$\Psi(\eta) \begin{cases} > 0, & \eta < \eta^*, \\ = 0, & \eta = \eta^*, \\ < 0, & \eta > \eta^*. \end{cases}$$

For a fixed η , define $G_i^s(r; \eta)$ as the maximum value of

$$I(M) - \eta aK(M)$$

over the first i channels, with r selected channels and final state s .

The initialization is $G_0^0(0; \eta) = 0$, with all other initial or unreachable states set to $-\infty$. The transitions are

$$G_i^0(r; \eta) = \max \{ G_{i-1}^0(r; \eta), G_{i-1}^1(r; \eta) \},$$

and

$$G_i^1(r; \eta) = v_i + \max \{ G_{i-1}^1(r-1; \eta), G_{i-1}^0(r-1; \eta) - \eta a \}.$$

Continuing a selected run adds no new chunk cost, whereas changing from state 0 to state 1 starts a new chunk and incurs the penalty ηa .

After processing all channels,

$$\Psi(\eta) = \max_{s \in \{0,1\}} G_N^s(R; \eta) - \eta cR.$$

Dinkelbach's method repeatedly solves this parametric chain problem and updates

$$\eta_{t+1} = \frac{I(M_t)}{aK(M_t) + cR},$$

where M_t maximizes the transformed objective at parameter η_t . The procedure terminates when

$$\Psi(\eta_t) = 0$$

in exact arithmetic. Numerical implementations use a stated residual tolerance.

Each parametric chain-DP evaluation takes

$$O(NR)$$

time. If Dinkelbach's method performs q iterations, the total running time is

$$\boxed{O(qNR),}$$

with $O(R)$ memory for values or $O(NR)$ with parent pointers for mask recovery. Here q is the actual number of iterations.

The three-dimensional dynamic program provides a direct polynomial-time exact oracle, while the parametric chain-DP removes the explicit chunk-count dimension and is potentially much faster in practice. With exact arithmetic, exact inner solves, and zero-residual termination for Dinkelbach, both methods solve the fitted affine objective exactly:

$$\frac{I(M)}{aK(M) + cR},$$

but this guarantee does not extend to the original lookup-table objective when $T[r]$ deviates from $a + cr$.

An importance-coverage constraint defines an alternative optimization problem.

Coverage formulation

The reconstructed fixed-budget formulation prescribes the number of selected rows,

$$R(M) = R,$$

and searches for a mask with a favorable importance–latency ratio. The alternative formulation reverses these roles: it prescribes how much activation importance must be retained and minimizes the latency required to retain it.

Define the total activation importance as

$$I_{\text{tot}} = \sum_{i=1}^N v_i,$$

and let $\alpha \in (0, 1]$ be the target importance-retention ratio. The required importance is

$$B_\alpha = \alpha I_{\text{tot}}.$$

Under the affine model, the proposed coverage problem is

$$M_\alpha^* \in \arg \min_M L_{\text{aff}}(M) \quad \text{subject to} \quad I(M) \geq \alpha I_{\text{tot}}.$$

The inequality allows coverage to exceed the target, since exact equality may be impossible. Among masks with minimum latency, choose one with the largest retained importance.

Unlike the fixed-budget problem, this formulation does not prescribe $R(M)$. Both the number of selected rows and the number of chunks are determined by the optimization. Concentrated importance may allow the target to be reached with relatively few rows, whereas a smoother importance distribution may require selecting more rows.

Exact offline solution

The chain dynamic program developed for the fixed-budget problem can also solve the affine coverage problem exactly. For every feasible pair (r, k) , define

$$I^*(r, k) = \max_{\substack{M \in \{0,1\}^N \\ R(M)=r \\ K(M)=k}} I(M) = \max_{s \in \{0,1\}} F_N^s(r, k).$$

Every mask with $R(M) = r$ and $K(M) = k$ has the same affine latency,

$$aK(M) + cR(M) = ak + cr.$$

Consequently, some mask with row count r and chunk count k satisfies the coverage constraint if and only if

$$I^*(r, k) \geq \alpha I_{\text{tot}}.$$

The optimal counts are therefore

$$(r_\alpha^*, k_\alpha^*) \in \arg \min_{\substack{0 \leq r \leq N \\ 0 \leq k \leq K_{\max}(r) \\ I^*(r, k) \geq \alpha I_{\text{tot}}}} (ak + cr),$$

where

$$K_{\max}(r) = \min(r, N - r + 1)$$

for $r \geq 1$, with $K_{\max}(0) = 0$.

Break cost ties between states by larger $I^*(r, k)$. The corresponding optimal mask can be recovered by backtracking through the DP state attaining (r_α^*, k_α^*) . Equivalently, the exact minimum latency is

$$L_{\text{aff}}^*(\alpha) = \min_{\substack{r, k \\ I^*(r, k) \geq \alpha I_{\text{tot}}}} (ak + cr).$$

Computing the complete table $I^*(r, k)$ requires

$$O(N^3)$$

time and $O(N^2)$ rolling memory for values. Direct backtracking with all parent pointers takes $O(N^3)$ memory; smaller-memory recovery requires recomputation. The final table can be reused across coverage thresholds.

This algorithm is therefore suitable as an exact offline oracle. It establishes the best achievable latency under the affine model, but its cubic running time is unlikely to satisfy the paper's approximately 2 ms per-matrix online-selection target.

A fast Lagrangian chain solver

A faster online approximation can be obtained by relaxing the coverage constraint with a multiplier $\lambda \geq 0$:

$$\min_M [aK(M) + cR(M) + \lambda (B_\alpha - I(M))].$$

Because λB_α is constant for a fixed multiplier, the mask can be found by solving

$$\min_M [aK(M) + cR(M) - \lambda I(M)].$$

The multiplier λ controls the value assigned to retained importance:

- a small λ emphasizes low latency;
- a large λ rewards retaining more importance.

For a fixed λ , define $D_i^s(\lambda)$ as the minimum relaxed cost for the first i channels when channel i has state $s \in \{0, 1\}$. The initial conditions are

$$D_0^0(\lambda) = 0, \quad D_0^1(\lambda) = +\infty.$$

If channel i is not selected,

$$D_i^0(\lambda) = \min \{D_{i-1}^0(\lambda), D_{i-1}^1(\lambda)\}.$$

If channel i is selected, it either continues the current chunk or starts a new one:

$$D_i^1(\lambda) = c - \lambda v_i + \min \{ D_{i-1}^1(\lambda), D_{i-1}^0(\lambda) + a \}.$$

The term $c - \lambda v_i$ is the marginal cost of selecting channel i . A transition from 0 to 1 additionally pays the per-chunk cost a , while a transition from 1 to 1 extends an existing chunk without paying a again.

This recurrence has only two states per channel. It does not explicitly track either $R(M)$ or $K(M)$. Therefore, one multiplier evaluation requires

$$O(N)$$

time. Evaluating q multiplier values requires

$$O(qN)$$

time, with $O(1)$ memory when only the optimal value is needed and $O(N)$ memory when the mask must be recovered.

A grid search or bisection-style search over λ can generate masks ranging from low-latency, low-importance solutions to higher-importance solutions. Among the generated masks satisfying

$$I(M) \geq \alpha I_{\text{tot}},$$

the mask with the smallest affine latency is returned, breaking ties by greater importance. Keep the full-selection mask as a feasible fallback if the searched multipliers produce no feasible candidate.

The $O(qN)$ complexity makes this method a plausible candidate for approaching the paper’s online runtime target. Unlike Neuron Chunking, it requires neither multiscale candidate generation nor global candidate sorting. Its practical latency must nevertheless be established experimentally rather than inferred from asymptotic complexity alone.

The Lagrangian solver is not guaranteed to recover the exact coverage optimum. Linear scalarization can recover only supported points of the discrete importance–latency frontier. The minimum-latency mask satisfying a particular coverage threshold may be an unsupported point and may therefore fail to minimize the relaxed objective for every value of λ .

The two algorithms consequently serve different purposes:

Method	Complexity	Role
Exact (r, k) chain DP	$O(N^3)$	Offline optimality oracle

Method	Complexity	Role
Lagrangian chain DP	$O(qN)$	Fast online approximation

The exact DP provides the ground truth against which the faster solver can be evaluated.

Importance–latency frontiers

Sweeping a finite set of coverage thresholds samples the exact affine importance–latency frontier

$$\left\{ \left(\frac{I(M_\alpha^*)}{I_{\text{tot}}}, L_{\text{aff}}^*(\alpha) \right) : \alpha \in \mathcal{A} \right\}.$$

A representative set of thresholds is

$$\mathcal{A} = \{0.80, 0.85, 0.90, 0.95, 0.97, 0.99\}.$$

If $\alpha_1 \leq \alpha_2$, every mask feasible at α_2 is also feasible at α_1 . Hence,

$$\alpha_1 \leq \alpha_2 \implies L_{\text{aff}}^*(\alpha_1) \leq L_{\text{aff}}^*(\alpha_2).$$

The curve therefore measures the minimum additional latency required to preserve progressively more importance. It does not imply that retaining more importance reduces latency.

The experiment can compare four sets of operating points on the same activation vectors:

- the exact coverage frontier obtained from the $O(N^3)$ DP;
- the $O(qN)$ Lagrangian solutions;
- the masks produced by Neuron Chunking;
- the masks produced by conventional top- R selection.

For any produced mask M , define its achieved coverage as

$$\alpha(M) = \frac{I(M)}{I_{\text{tot}}}.$$

For $I(M) > 0$, its relative gap under the affine model is

$$\text{Gap}_{\text{aff}}(M) = \frac{L_{\text{aff}}(M) - L_{\text{aff}}^*(\alpha(M))}{L_{\text{aff}}^*(\alpha(M))}.$$

Using the achieved coverage $\alpha(M)$, rather than the requested threshold, ensures that each method is compared with the exact optimum preserving at least as much importance as that method actually

preserves.

Recording

$$R(M) \quad \text{and} \quad K(M)$$

at each operating point additionally reveals whether a latency reduction comes from selecting fewer rows, forming fewer chunks, or both.

The fixed- R and coverage frontiers answer distinct questions:

Frontier	Constraint	Variables allowed to change	Primary purpose
Fixed- R frontier	$R(M) = R$	$K(M)$ and selected positions	Evaluate the problem suggested by the paper’s implementation
Coverage frontier	$I(M) \geq \alpha I_{\text{tot}}$	$R(M)$, $K(M)$, and selected positions	Evaluate the alternative problem proposed here

For each mask, record L_{aff} , L_{table} , and L_{measured} separately to distinguish optimization quality from model accuracy. Use the same channel order and reordering across methods, and include activation transfers, mask recovery, and synchronization in selection time.

Finally, activation importance remains a surrogate for model quality. Exact optimality on the importance–latency frontier does not imply exact optimality on the downstream accuracy–latency frontier. Selected operating points must therefore also be evaluated on the original VLM tasks.